

Organisation Front + Back — textes affichés et réactions des boutons

Vue d'ensemble

FRONT (React)

BACK (Symfony)

layouts/AppLayout.tsx ← texte menu global

features/*/Page.tsx ← textes page + handlers

components/* ← textes blocs réutilisables

constants/ + utils/ ← libellés statuts, boutons bulle

api/*.ts → Controller/Api/*.php

client.ts → Service/*.php → messages MetierException

→ ApiPresenter.php → données JSON (noms, motifs...)

PARTIE 1 — FRONTEND

Arborescence utile

frontend/src/

└─ main.tsx → point d'entrée React

└─ App.tsx → BrowserRouter

└─ routes/AppRoutes.tsx → /usager, /reception, /personnel

└─ layouts/AppLayout.tsx → en-tête, menu, pied de page (textes globaux)

|

└─ features/ → PAGES (écrans complets)

| └─ usager/UsagerPage.tsx

| └─ reception/

| | └─ ReceptionPage.tsx → mise en page + branchement bulle

| | └─ useReceptionPage.ts → état + handlers + appels API

| | └─ renderReceptionBubble.tsx

| └─ personnel/PersonnelPage.tsx

|

└─ components/ → morceaux d'UI (textes + boutons locaux)

| └─ reception/ → bulle, bannière, panneau visite

- | |— appointments/ → table / cartes RDV
- | |— visits/ → DecisionActions, VisitCard
- | |— notifications/, historique/, ui/, usager/, personnel/
- |
- |— api/ → PAS de texte utilisateur : URLs + fetch
- |— constants/ → libellés statuts, suggestions fonctions
- |— utils/ → textes « action suivante », alertes fonction
- |— types/api.ts → types TypeScript (pas de texte UI)
- |— index.css → couleurs institutionnelles (pas de phrases)

_legacy/ : anciens mocks — **non utilisés**, pas dans l'app actuelle.

Où sont écrits les mots à l'écran (front)

A) Textes fixes (vous les avez codés en dur)

Zone	Fichier	Exemples
Menu + titre site	layouts/AppLayout.tsx	« Centre d'accueil — C2I », « Usager », « Réception », footer MVP
Page Usager	features/usager/UsagerPage.tsx	titres sections, labels champs, bouton « Envoyer la demande », écran confirmation
Page Réception	features/reception/ReceptionPage.tsx	« Accueil — réception », « Messages reçus », « Rendez-vous du jour »
Page Personnel	features/personnel/PersonnelPage.tsx	titres, boutons dispo, démarrer/clôturer
Bulle actions	components/reception/ReceptionActionBubble.tsx	« Action suivante », « Fermer », « Voir l'historique »
Boutons décision	components/visits/DecisionActions.tsx	« Orienter », « Attendre », « Clôturer », « Reporter », « Réorienter »

Zone	Fichier	Exemples
Période usager	components/usager/PreferencePeriodePicker.tsx	Matin / Après-midi
Barre recherche	components/ui/SearchBar.tsx	placeholder (si défini dans le composant)

B) Textes centralisés (un fichier → plusieurs écrans)

Fichier	Rôle
constants/status.ts	Badges : « Prévu », « Arrivé », « Demande à traiter »...
utils/receptionQuickActions.ts	Libellés bulle : « Enregistrer l'arrivée », « Ouvrir la visite », boutons selon statut
utils/receptionGuidance.ts	Bannière « Action suivante » (titre + détail court)
utils/rendezVousDisplay.ts	Alerte : « La personne recherchée n'est pas disponible... »
constants/fonctionsSouhaitees.ts	Suggestions datalist usager (Technicien, etc.)
components/notifications/NotificationPanel.tsx	« Prise en charge terminée », « Décision attendue »

C) Textes venant de l'API (backend / BDD)

Donnée affichée	Source JSON	Exemple
Nom usager, téléphone	rdv.usager	fixtures ou formulaire
Nom bureau	rdv.bureau.nom	DemoFixtureLoader
Motif RDV	rdv.motif	saisi à la création
Référence RDV	rdv.reference	générée backend (ReferenceGenerator)
Message d'erreur rouge	error dans JSON	MetierException côté PHP
Texte notification	notification.message	NotificationService

Donnée affichée	Source JSON	Exemple
Historique	historique[].libelle	HistoriqueService

Le front **ne invente pas** le nom « Bureau Secrétariat » : il l’affiche tel que renvoyé par ApiPresenter.

Réactions des boutons (front) — qui fait quoi

Chaîne générale

Clic bouton

- handler dans Page ou useReceptionPage
- fonction dans api/reception.ts (ou rendezVous, personnel...)
- client.ts (fetch)
- API Symfony
- ← data ou ApiClientError
- setState (liste, erreur, confirmation)
- React réaffiche

Page Usager (UsagerPage.tsx)

Action utilisateur	Handler	API appelée
Chargement page	loadBureaux()	GET /api/bureaux
Submit formulaire	handleSubmit	POST /api/rendez-vous via createRendezVous()
Erreur	catch → setSubmitError	message ApiClientError (= json.error serveur)
Succès	setConfirmation({ rdv })	écran confirmation (textes dans la même page)

Textes confirmation : dans UsagerPage.tsx (« Rendez-vous planifié », « Demande enregistrée »).

Page Réception — cœur du flux

Fichier	Rôle
ReceptionPage.tsx	Affichage + handleQuickAction (mappe id bouton → handler)

Fichier	Rôle	
useReceptionPage.ts	Toute la logique : chargement listes, recherche, actions	
renderReceptionBubble.tsx	Instancie ReceptionActionBubble	
ReceptionActionBubble.tsx	Boutons UI ; onClick → onAction(id)	
Clic (bulle ou panneau)	Handler (useReceptionPage)	API
Enregistrer l'arrivée	handleArrivee	POST /api/reception/rendez-vous/{id}/arrivee
Ouvrir la visite	handleOuvrirVisite	POST /api/reception/visites
Orienter	handleOrienter	PATCH .../visites/{id}/orienter
Mettre en attente	handleDecision('ATTENDRE')	PATCH .../decision
Clôturer visite	handleCloturerVisite	PATCH .../cloturer
Reporter / Réorienter	handleDecision('REPORTER'/'REORIENTER')	idem
Clic ligne tableau	selectRdv	pas d'API — ouvre bulle
Recherche	handleSearch	GET /api/reception/recherche?query=
Chargement initial	loadRegistre	GET rendez-vous-du-jour + rendez-vous-a-venir

Après chaque action : runAction → refreshLists() + loadNotifications() → la bulle se met à jour.

Page Personnel (PersonnelPage.tsx)

Clic	Handler	API
Choisir un agent	handleSelectPersonnel	GET /api/personnel/{id}/rendez-vous
Disponible / Occupé / Ne peut pas recevoir	setDispo(...)	PATCH /api/personnel/{id}/disponibilite

Clic	Handler	API
Démarrer	onDemarrer	PATCH .../rendez-vous/{id}/demarrer
Clôturer	onCloturer	PATCH .../cloturer (+ notification réception)

Fichiers API front (pas de texte bouton, seulement les URLs)

Fichier	Routes utilisées
api/client.ts	fetch + parseJson + messages génériques (« Impossible de joindre le serveur »)
api/bureaux.ts	/api/bureaux
api/rendezVous.ts	/api/rendez-vous
api/reception.ts	tout /api/reception/...
api/personnel.ts	/api/personnel/...
api/notifications.ts	notifications
api/historique.ts	historique visite

PARTIE 2 — BACKEND

Arborescence utile

backend/src/

```

├── Controller/Api/    → routes HTTP (peu de texte : erreurs validation)
|   ├── AbstractApiController.php → jsonOk / jsonError
|   ├── RendezVousController.php
|   ├── ReceptionController.php
|   ├── PersonnelController.php
|   ├── BureauController.php
|   ├── NotificationController.php
|   └── HistoriqueController.php
|

```

- └— Service/ → règles métier + messages d'erreur MetierException
- └— Entity/ → structure BDD (pas de texte UI)
- └— Enum/ → valeurs techniques (CONFIRME, ATTENDRE...)
- └— DTO/ → champs reçus du JSON (CreateRendezVousDto...)
- └— Presenter/ApiPresenter.php → forme du JSON renvoyé au front
- └— Repository/ → requêtes SQL/Doctrine
- └— DataFixtures/ → données DEMO (noms, motifs, téléphones A-F)
- └— Exception/MetierException.php
- └— Util/DtoMapper.php, ReferenceGenerator.php

Où sont les mots côté back

Type de texte Où

Erreurs affichées en rouge sur le front throw new MetierException('...') dans Service/*.php → AbstractApiController::handle() → { success: false, error: "..." }

Données métier (noms, bureaux, motifs) DataFixtures/DemoFixtureLoader.php (démon) ou créées par RendezVousService à la soumission usager

Messages notifications NotificationService (ex. fin de prise en charge)

Libellés historique HistoriqueService (ex. « Arrivée enregistrée », « Visite ouverte »)

Format JSON (clés, pas phrases UI) Presenter/ApiPresenter.php

Le backend **ne pilote pas** les libellés « Prévu / Arrivé » des badges : il envoie statut: "CONFIRME" ; le front traduit via constants/status.ts.

Réactions des actions (back) — route → service

Route	Controller	Service principal	Effet
POST /api/rendez-vous	RendezVousController::create	RendezVousService::creerDemande + CreneauService	Crée RDV CONFIRME ou DEMANDE
POST .../arrivee	ReceptionController::arrivee	ReceptionService::enregistrerArrivee	Statut → ARRIVE
POST /api/reception/visites	ouvrirVisite	ReceptionService::ouvrirVisite	Crée visite
PATCH .../orienter	orienter	ReceptionService::orienterUsager	Lie personnel, visite ORIENTEE
PATCH .../decision	decision	ReceptionService::deciderSuiteVisite	ATTENDRE, REPORTER, etc.
PATCH .../cloturer	cloturer	ReceptionService::cloturerVisite	Visite terminée
PATCH .../demarrer	PersonnelController	PersonnelService	RDV EN_COURS
PATCH .../cloturer (personnel)	idem	PersonnelService	RDV TERMINE + notification

Tableau croisé : bouton → fichier front → API → fichier back

Bouton (texte front)	Texte écrit dans	Handler front	Route API	Service back
Envoyer la demande	UsagerPage.tsx	handleSubmit	POST /api/rendez-vous	RendezVousService
Enregistrer l'arrivée	receptionQuickActions.ts	handleArrivee	POST .../arrivee	ReceptionService
Ouvrir la visite	idem	handleOuvrirVisite	POST /api/reception/visites	ReceptionService
Orienter	DecisionActions / bulle	handleOrienter	PATCH .../orienter	ReceptionService
Clôturer la visite	idem	handleCloturerVisite	PATCH .../cloturer	ReceptionService
Démarrer (personnel)	PersonnelPage.tsx	onDemarrer	PATCH .../demarrer	PersonnelService
Clôturer (personnel)	idem	onCloturer	PATCH .../cloturer	PersonnelService

Pour retrouver un mot précis

1. **Texte fixe français** → recherche dans frontend/src (Ctrl+F sur la phrase).
2. **Badge statut** → constants/status.ts ou receptionQuickActions.ts.
3. **Message d'erreur rouge** → souvent backend : chercher la phrase dans backend/src/Service.
4. **Nom d'utilisateur / bureau en liste** → fixtures DemoFixtureLoader.php ou données créées via formulaire usager.
5. **Comportement d'un bouton** → page features/.../Page.tsx ou useReceptionPage.ts → fichier api/*.ts → Controller/Api → Service.

Phrase pour le prof

« Les **libellés d'interface** sont dans les composants React et les fichiers constants/ / utils/. Les **réactions** passent par des hooks (useReceptionPage) qui appellent api/*.ts puis client.ts. Le **backend** expose des routes dans Controller/Api, applique les règles dans Service, renvoie

les **données** via *ApiPresenter* et les **erreurs métier** via *MetierException* que le front affiche avec *parseJson / ApiClientError*. »

Si vous voulez une **fiche par page** (liste exhaustive de chaque label du formulaire usager ligne par ligne), indiquez la page (/usager, /reception ou /personnel) et je la détaille.